Reviewer Pack — Minimal Local Induction Dimension of Affine Schemes and DPLL…

Entry 56c241670c2a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-11 18:30:15 UTC

Minimal Local Induction Dimension of Affine Schemes and DPLL Search Tree Width

Entry ID: `56c241670c2a` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-11 18:30:15 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry - Affine Schemes **Field B** (complexity object): Boolean Satisfiability Problem - DPLL Search Tree

Statement:

> For every Boolean satisfiability instance φ with n variables, the minimal local induction dimension (mld(φ)) of its associated affine scheme is linearly correlated with its DPLL search tree width w_DPLL(φ), such that mld(φ) = Θ(w_DPLL(φ)).

Rationale (proposer's reasoning):

> Affine schemes provide a geometric interpretation of Boolean functions, and their local induction dimension captures the complexity of these functions. By analyzing this dimension in the context of DPLL search trees, we may uncover new insights into the inherent difficulty of satisfiability problems.

Taxonomy category: `affine_schemes_to_dpll` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `3948ff85ebae9070`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the computed minimal local induction dimension (mld) of the affine scheme for all Boolean satisfiability instances φ satisfies |mld(φ) - w_DPLL(φ)| ≤ 3, where mld and w_DPLL are computed for n variables with a linear correlation coefficient ≥ 0.8 over at least 30 seeds.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal local induction dimension" AND "affine schemes" AND "DPLL search tree width" - "mld affine scheme" AND "Boolean satisfiability problem" AND DPLL - "correlation mld affine scheme" AND Boolean AND "satisfiability problem"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.4s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_instance(n):
    return [random.choice([0, 1]) for _ in range(2**n)]

def dpll_width(instance):
    n = len(instance)
    if n == 0:
        return 0
    if all(x == 0 or x == 1 for x in instance):
        return 1

    # Find a literal to branch on
    literals = [i for i, x in enumerate(instance) if x != 2]
    if not literals:
        return 1

    lit = random.choice(literals)

    positive_clauses = []
    negative_clauses = []
    for clause in instance:
        if clause[lit] == 0:
            negative_clauses.append(clause[:lit] + [2] + clause[lit+1:])
        elif clause[lit] == 1:
            positive_clauses.append(clause[:lit] + [2] + clause[lit+1:])

    return max(dpll_width(positive_clauses), dpll_width(negative_clauses))

def run_trial(seed: int) -> dict:
    random.seed(seed)
```

```
    n_values = [5, 10, 15, 20, 30, 40]
    mld_values = []
    w_dpll_values = []

    for n in n_values:
        instances = [generate_instance(n) for _ in range(30)]
        mld_values.extend([sum(x != 2 for x in instance) for instance in instances])
        w_dpll_values.extend([dpll_width(instance) for instance in instances])

    if not mld_values or not w_dpll_values:
        return {
            "metric_name": "mld_w_dpll_correlation",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": max(n_values),
            "conjecture_holds": False,
            "counterexample": "empty_instance"
        }

    mld_mean = sum(mld_values) / len(mld_values)
    w_dpll_mean = sum(w_dpll_values) / len(w_dpll_values)
    correlation_coefficient = sum((m - mld_mean) * (w - w_dpll_mean) for m, w in zip(mld_values, w

    return {
        "metric_name": "mld_w_dpll_correlation",
        "metric_value": correlation_coefficient,
        "instances_tested": len(mld_values),
        "n_max": max(n_values),
        "conjecture_holds": abs(correlation_coefficient) >= 0.8 and all(abs(m - w) <= 3 for m, w i
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{trial_result['metric_name']}\", \"m
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean = sum(r["metric_value"] for r in results) / len(results)
        std_dev = math.sqrt(sum((r["metric_value"] - mean)**2 for r in results) / len(results))
        support_fraction = 1.0
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        mean = None
        std_dev = None
        support_fraction = sum(result["conjecture_holds"] for result in results) / len(results)

    print(f"RESULT: {'SUPPORTED' if all(result['conjecture_holds'] for result in results) else 'FA
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, which means that the pre-registered support conditions could not be verified. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	27949
2	preregistration	ollama_remote	glm4:latest	0	0	13149
3	novelty	ollama_remote	glm4:latest	0	0	8386
4	novelty	ollama_remote	glm4:latest	0	0	8724
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17564
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13315
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11875
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11914
9	judge	ollama_remote	glm4:latest	0	0	22117

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 134992 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/56c241670c2a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/56c241670c2a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/56c241670c2a.tar.gz` (if generated)